

Softwarekonzept – RehaPlaner+

Inhaltsverzeichnis

1	Einleitung	3
2	Zielsetzung und Use Cases.....	3
2.1	Zielsetzung.....	3
2.2	Haupt-Use-Cases.....	3
3	Systemübersicht.....	4
3.1	Systemkontext.....	4
3.2	Grobarchitektur.....	5
4	Fachliche Anforderungen.....	5
4.1	PDF-Parsing und Terminextraktion	5
4.2	Dokumentenverwaltung.....	6
4.3	Chat-Funktionalität	6
4.4	Nutzerverwaltung und Login.....	6
5	Nicht-funktionale Anforderungen	7
6	Systemarchitektur und Komponenten.....	7
6.1	Komponentenübersicht.....	7
6.2	Datenfluss (Beispiel UC1 – Behandlungsplan hochladen).....	8
7	Datenmodell (vereinfacht)	8
8	Sicherheit und Datenschutz (Zielbild)	10
9	Abgrenzung und Scope	10
9.1	Geplante, aber (noch) nicht vollständig umgesetzte Funktionen	10
9.2	Nicht Bestandteil des aktuellen Konzepts	11
10	KI-Verzeichnis	11

1 Einleitung

Dieses Softwarekonzept beschreibt die geplante Architektur, Funktionalität und Rahmenbedingungen eines webbasierten Systems zur Unterstützung des Reha-Managements. Im Fokus stehen:

- Auslesen von Reha-Dokumenten (insbesondere Behandlungspläne) mittels PDF-Parsing
- Strukturierte Darstellung der Termine und Inhalte
- Kommunikation über einen Chat zwischen mehreren Beteiligten
- Nutzung echter Reha-Dokumente (im Zielsystem)
- Authentifizierung und rollenbasierter Zugriff auf die Daten

Das Konzept beschreibt den Soll-Zustand des Systems. Abweichungen aufgrund von Scope-Begrenzungen in der Umsetzung werden in einem eigenen Abschnitt „Umsetzungsstand vs. Konzept“ festgehalten.

2 Zielsetzung und Use Cases

2.1 Zielsetzung

Ziel ist die Entwicklung eines Prototyps für ein Reha-Informationssystem, das:

- Behandlungspläne aus PDF-Dokumenten automatisiert ausliest
- Termine und Maßnahmen strukturiert und übersichtlich darstellt
- Kommunikation zwischen relevanten Akteuren (z. B. Patient, Therapeut, Verwaltung) über einen Chat ermöglicht
- Perspektivisch echte Reha-Dokumente statt Mockups verarbeitet
- Zugriff nur für authentifizierte Nutzer mit passenden Rollen erlaubt

2.2 Haupt-Use-Cases

- **UC1 – Behandlungsplan hochladen:** Nutzer lädt ein PDF mit einem Reha-Behandlungsplan hoch, das System extrahiert Termine und Maßnahmen.

- **UC2 – Termine anzeigen:** Nutzer sieht eine strukturierte Liste/Kalenderansicht der ausgelesenen Termine (Datum, Uhrzeit, Maßnahme, Ort, ggf. Verantwortliche).
- **UC3 – Dokumente verwalten:** Nutzer kann hochgeladene Dokumente einsehen, ggf. löschen oder neu hochladen.
- **UC4 – Chat nutzen:** Nutzer kann Nachrichten in einem Chat schreiben und empfangen (geplant: mehrere Teilnehmer, z. B. Patient, Therapeut, Verwaltung).
- **UC5 – Login & Profil:** Nutzer meldet sich mit Zugangsdaten an und nutzt ein persönliches (im Prototyp: Mockup-)Profil.

3 Systemübersicht

3.1 Systemkontext

Akteure:

- **Patient** (oder Demo-Nutzer): lädt Dokumente hoch, sieht Termine, nutzt Chat.
- **Therapeut / Fachpersonal** (geplant): kann Termine einsehen, kommentieren, im Chat kommunizieren.
- **Verwaltung / Admin** (geplant): verwaltet Nutzer, Rollen und ggf. Dokumente.

Externe Systeme (perspektivisch):

- **Reha-Klinik-Systeme** (z. B. KIS): könnten später als Datenquelle für echte Dokumente dienen.
- **Authentifizierungsdienst** (z. B. OAuth/SSO): für produktive Umgebungen denkbar.

3.2 Grobarchitektur

- **Frontend:** Web-Client (z. B. SPA mit React/Vue/Angular oder klassisches Web-Frontend), Darstellung von Dokumenten, Terminen, Chat und Login.
- **Backend:** REST- oder GraphQL-API, die:
 - PDF-Uploads entgegennimmt
 - Parsing anstößt
 - Daten in einer Datenbank speichert
 - Chat-Nachrichten verwaltet
 - Authentifizierung/Autorisierung prüft
- **Datenbank:** Relationale oder dokumentenbasierte DB zur Speicherung von:
 - Nutzern & Rollen
 - Dokumenten-Metadaten
 - Extrahierten Terminen
 - Chat-Nachrichten

4 Fachliche Anforderungen

4.1 PDF-Parsing und Terminextraktion

☐ **FA1 – PDF-Upload:** Nutzer kann ein PDF-Dokument (Behandlungsplan) hochladen.

☐ **FA2 – Automatische Extraktion:** Das System versucht, alle Termine aus dem Behandlungsplan zu extrahieren:

- Datum
- Uhrzeit
- Bezeichnung der Maßnahme
- ggf. Ort / Raum
- ggf. verantwortliche Person

☐ **FA3 – Fehlerrobustheit:** Bei komplexen oder uneinheitlichen PDF-Strukturen kann es zu unvollständiger Extraktion kommen. Im Konzept ist das Ziel, möglichst alle Termine zu erfassen; in der aktuellen

Umsetzung werden z. B. nur ca. 74 von 100–120 Terminen erkannt (Parsing-Limitierung).

□ **FA4 – Anzeige der Extraktion:** Die extrahierten Termine werden dem Nutzer in einer strukturierten Ansicht (Liste oder Kalender) angezeigt.

4.2 Dokumentenverwaltung

- **FA5 – Dokumentübersicht:** Nutzer sieht eine Liste seiner hochgeladenen Dokumente mit Metadaten (Name, Upload-Datum, Status der Extraktion).
- **FA6 – Dokumentdetails:** Zu jedem Dokument können die zugehörigen extrahierten Termine eingesehen werden.
- **FA7 – Echte Reha-Dokumente (Ziel):** Im Zielsystem sollen echte Reha-Dokumente verwendet werden, um realistische Strukturen und Inhalte abzubilden. Im Prototyp können aus Datenschutzgründen Mockup-Dokumente verwendet werden.

4.3 Chat-Funktionalität

- **FA8 – Chat zwischen mehreren Personen (Ziel):** Geplant ist ein **Mehrpersonen-Chat**, in dem z. B. Patient, Therapeut und Verwaltung miteinander kommunizieren können.
- **FA9 – Nachrichtenverwaltung:** Nachrichten werden persistent gespeichert und sind nachträglich einsehbar.
- **FA10 – Zuordnung zu Kontext:** Perspektivisch können Chats an einen Patienten, einen Behandlungsplan oder bestimmte Termine gekoppelt werden.

(Hinweis: Im Prototyp kann der Chat ggf. nur eingeschränkt oder als Single-User-Demo umgesetzt sein, da Mehrpersonen-Chat den Scope sprengt.)

4.4 Nutzerverwaltung und Login

- **FA11 – Authentifizierung:** Nutzer melden sich mit Login-Daten an (E-Mail/Benutzername + Passwort).

- **FA12 – Rollen & Rechte (Ziel):** Rollen wie Patient, Therapeut, Admin steuern, welche Daten ein Nutzer sehen und bearbeiten darf.
- **FA13 – Mockup-Profil:** Im Prototyp existiert ein Mockup-Profil, das beispielhafte Daten anzeigt (Name, Rolle, ggf. Basisinformationen).

5 Nicht-funktionale Anforderungen

- **NFA1 – Usability:** Einfache, intuitive Bedienung; klare Navigation zwischen Dokumenten, Terminen und Chat.
- **NFA2 – Performance:** PDF-Parsing darf einige Sekunden dauern, sollte aber den Nutzer über Ladezustände informieren.
- **NFA3 – Sicherheit:**
 - Geschützter Zugriff auf Dokumente und Termine
 - Passwörter nur gehasht speichern
 - Transportverschlüsselung (HTTPS) im Zielsystem
- **NFA4 – Datenschutz:** Bei echten Reha-Dokumenten sind Datenschutzanforderungen (z. B. DSGVO) zu beachten; im Prototyp ggf. nur Pseudodaten/Mockups.
- **NFA5 – Wartbarkeit:** Klare Trennung von Frontend, Backend, Parsing-Logik und Datenzugriff.

6 Systemarchitektur und Komponenten

6.1 Komponentenübersicht

Frontend-Komponente:

- Login-Maske
- Dashboard (Dokumentenliste, Termine, Chat)
- Detailansichten (Dokumentdetails, Terminliste, Chatverlauf)

Backend-Komponente:

- Auth-Service: Login, Token-Verwaltung
- Document-Service: Upload, Speicherung, Metadaten

- Parsing-Service: PDF-Analyse, Extraktion der Termine
- Schedule-Service: Verwaltung der extrahierten Termine
- Chat-Service: Nachrichtenverwaltung, ggf. WebSocket-Anbindung
- User-Service: Nutzer- und Rollenverwaltung

Datenbank-Komponente:

- Tabellen/Collections für:
 - Nutzer
 - Rollen
 - Dokumente
 - Termine
 - Chat-Nachrichten

6.2 Datenfluss (Beispiel UC1 – Behandlungsplan hochladen)

- Nutzer lädt PDF hoch im Frontend.
- Frontend sendet Datei an Backend (Document-Service).
- Document-Service speichert Datei (z. B. im Filesystem oder Object Storage) und legt Metadaten in der DB an.
- Document-Service ruft Parsing-Service auf.
- Parsing-Service analysiert das PDF, extrahiert Termine und gibt strukturierte Daten zurück.
- Schedule-Service speichert die Termine in der Datenbank.
- Frontend ruft die Termine ab und zeigt sie dem Nutzer an.

7 Datenmodell (vereinfacht)

□ User

- id
- username / email
- passwordHash

- role (Patient, Therapeut, Admin, ...)

□ **Document**

- id
- userId
- filename
- uploadDate
- parsingStatus (pending, success, partial, failed)

□ **Appointment (Termin)**

- id
- documentId
- date
- time
- title (Maßnahme)
- location (optional)
- responsiblePerson (optional)

□ **ChatMessage**

- id
- conversationId (oder patientId/documentId)
- senderUserId
- timestamp
- content

□ **Conversation** (optional, für Mehrpersonen-Chat)

- id
- type (z. B. patient-based, document-based)
- participants (Liste von userIds)

8 Sicherheit und Datenschutz (Zielbild)

- **Authentifizierung:** Token-basierte Authentifizierung (z. B. JWT) für API-Zugriffe.
- **Autorisierung:** Rollenbasierte Zugriffskontrolle (RBAC), z. B.:
 - Patient sieht nur eigene Dokumente und Termine
 - Therapeut sieht nur zugewiesene Patienten
 - Admin hat erweiterte Rechte
- **Datenschutz:**
 - Minimierung personenbezogener Daten im System
 - Logging ohne sensible Inhalte
 - Bei echten Reha-Dokumenten: Einhaltung der DSGVO, Auftragsverarbeitungsverträge etc.

9 Abgrenzung und Scope

9.1 Geplante, aber (noch) nicht vollständig umgesetzte Funktionen

- **Vollständige Terminextraktion:** Ziel: nahezu alle Termine aus komplexen Behandlungsplänen erkennen. Aktueller Stand: ca. 74 von 100–120 Terminen werden erkannt, da die PDF-Struktur komplex ist.
- **Mehrpersonen-Chat:** Ziel: Chat zwischen mehreren Rollen (Patient, Therapeut, Verwaltung). Aktueller Stand: Aufgrund des Scope wurde der Chat nur eingeschränkt oder vereinfacht umgesetzt.
- **Echte Reha-Dokumente:** Ziel: Verarbeitung realer Reha-Dokumente. Aktueller Stand: Aus Datenschutz-/Organisationsgründen ggf. nur Mockup-Dokumente im Prototyp.
- **Vollständige Rollen- und Rechteverwaltung:** Ziel: Feingranulare Rechte pro Rolle. Aktueller Stand: Vereinfachtes Login mit Mockup-Profil.

9.2 Nicht Bestandteil des aktuellen Konzepts

- Integration in reale Klinik-Informationssysteme
- Mobile App als native Anwendung (iOS/Android)
- Vollständige Terminplanung (Erstellung/Änderung von Terminen im System statt nur Anzeige)

10 KI-Verzeichnis

Im Rahmen der Entwicklung der RehaPlaner+ PWA habe ich KI-Tools zunächst als Grundgerüst-Generator und später schrittweise als Debugging- und Unterstützungswerkzeug eingesetzt. Ich habe dabei ChatGPT 5.2 und GitHub Copilot 5.1 abwechselnd genutzt. Die KI-Unterstützung wurde nicht blind übernommen, sondern diente als Sparringspartner, Ideengeber und Debug-Hilfe. Alle finalen Entscheidungen, Anpassungen und Implementierungen wurden von mir selbst vorgenommen.

Nachfolgend dokumentiere ich die wichtigsten KI-Einsätze inklusive der Prompts, die ich verwendet habe.

1. Einsatz von ChatGPT 5.2 (OpenAI)

1.1 Architektur, Strukturierung & Konzeptentwicklung

Ich nutzte ChatGPT 5.2, um ein erstes Grundgerüst für die Projektarchitektur zu entwickeln.

Beispiel-Prompt:

„Ich entwickle eine Offline-fähige PWA, die PDF-Behandlungspläne per OCR ausliest. Bitte schlage mir eine sinnvolle Architektur vor (Frontend, Backend, Parsing-Service, Datenbank). Berücksichtige Offline-First, IndexedDB und Service Worker.“

Antworten wurden stark überarbeitet und an meine tatsächliche Implementierung angepasst.

1.2 Unterstützung bei fachlichen Anforderungen

ChatGPT half mir, die Anforderungen klar zu strukturieren.

Beispiel-Prompt:

„Formuliere fachliche Anforderungen für ein System, das PDF-Behandlungspläne automatisch in Termine umwandelt. Berücksichtige Extraktion, Fehlerrobustheit und Anzeige.“

Nur als sprachliche Inspiration genutzt, Inhalte wurden selbst ausgearbeitet.

1.3 Debugging der PDF-OCR-Pipeline

Ich nutzte ChatGPT, um Timing-Probleme zwischen PDF.js und Tesseract besser zu verstehen.

Beispiel-Prompt:

„Hier ist mein Code für den PDF-Upload und die OCR-Pipeline. Warum ist pdfjsLib manchmal undefined und warum liefert Tesseract leere Strings? Bitte analysiere die Race Conditions.“

ChatGPT gab Hinweise auf Script-Reihenfolge und Worker-Initialisierung, die ich manuell umgesetzt habe.

1.4 Regex-Optimierung für Terminparser

ChatGPT half mir, komplexe Regex-Varianten zu entwerfen.

Beispiel-Prompt:

„Ich habe OCR-Text aus einem Reha-Behandlungsplan. Bitte hilf mir, ein robustes Regex-Pattern zu entwickeln, das Uhrzeiten wie 09:00 - 09:30 erkennt, auch wenn Whitespace oder OCR-Fehler auftreten.“

Die generierten Patterns dienten als Ausgangspunkt, wurden aber stark überarbeitet.

1.5 Unterstützung bei der Formulierung des Softwarekonzepts

ChatGPT half bei der sprachlichen Strukturierung.

Beispiel-Prompt:

„Ich brauche ein professionelles Softwarekonzept für eine Reha-PWA. Bitte strukturiere die Kapitel logisch und formuliere präzise, aber nicht zu technisch.“

Ich habe die Struktur übernommen, Inhalte aber selbst ausgearbeitet.

2. Einsatz von GitHub Copilot 5.1 (Microsoft)

2.1 Generierung des initialen PWA-Grundgerüsts

Copilot half beim Erstellen der ersten Dateien.

Beispiel-Prompt im Code-Editor:

„Erstelle mir eine minimalistische Service-Worker-Datei für eine Offline-fähige PWA mit Cache-First-Strategie.“

Ich habe die generierten Dateien anschließend komplett überarbeitet.

2.2 Unterstützung beim Schreiben der PDF-Upload-Pipeline

Copilot schlug Codefragmente für die OCR-Pipeline vor.

Beispiel-Prompt:

„Ich möchte jede PDF-Seite rendern und per Tesseract erkennen. Ergänze meinen Code um eine Schleife über alle Seiten.“

Copilot-Vorschläge wurden angepasst und debuggt.

2.3 Debugging von IndexedDB-Transaktionen

Copilot half bei der Strukturierung der asynchronen DB-Operationen.

Beispiel-Prompt:

„Warum wird meine IndexedDB-Transaktion nicht abgeschlossen? Hier ist mein Code. Bitte schlage eine robuste put/getAll-Struktur vor.“

Ich habe die Logik manuell korrigiert und erweitert.

2.4 UI-Struktur & HTML-Grundgerüst

Copilot generierte erste UI-Layouts.

Beispiel-Prompt:

„Erstelle ein responsives HTML-Layout mit Bottom-Navigation, Chat-Bereich und Terminliste.“

Nur als Startpunkt genutzt, finaler Code komplett selbst geschrieben.

2.5 Unterstützung bei Regex-Fehleranalyse

Copilot half beim Erkennen von Fehlern in meinen Patterns.

Beispiel-Prompt:

„Warum matcht dieses Regex nicht bei OCR-Text? Bitte analysiere das Pattern und schlage eine robustere Variante vor.“

Copilot-Vorschläge dienten als Inspiration, wurden aber stark modifiziert.